

Game Design Document

Jose Manuel Iglesias Molina

Proyecto: Microjuego Asteroids

Motor: Unity

Plataforma: PC

ÍNDICE

1.	VISIÓN GENERAL Y ESCENAS.....	1
2.	MECÁNICAS E INTERACCIÓN.....	1
3.	ASSETS Y GAMEOBJECTS.....	2
4.	ARQUITECTURA DE SCRIPTS.....	2

1. VISIÓN GENERAL Y ESCENAS

1.1 High Concept

Juego arcade de disparos espaciales en 2D donde el jugador debe sobrevivir a oleadas de asteroides que aparecen desde cualquier límite de la pantalla. Al ser destruidos, los asteroides de mayor tamaño se fragmentan en pedazos más pequeños, multiplicando el número de objetivos en pantalla.

1.2 Diseño de Escenas

El proyecto se compone de una única escena principal (SampleScene) para optimizar el rendimiento y evitar tiempos de carga, dividida en dos capas lógicas:

- Capa de Gameplay ($Z = -1$): Todo el espacio interactivo (Nave, Asteroides, Fragmentos y Balas) está unificado estrictamente en la coordenada $Z = -1$. Esto garantiza colisiones perfectas y evita fugas en el eje de profundidad 3D al usar físicas.
- Capa de Interfaz (UI/Canvas): Renderizada en Screen Space-Overlay. Contiene el HUD (Score) y el Menú de Pausa.

2. MECÁNICAS E INTERACCIÓN

2.1 Bucle de Juego

1. **Aparición:** El sistema calcula los márgenes de la cámara (*orthographicSize*) y genera asteroides aleatoriamente desde los 4 bordes de la pantalla.

2. **Acción:** El jugador maniobra y dispara. La cadencia y velocidad de los enemigos aumenta con el tiempo (*spawnRateIncrement*)

3. Resolución de Impactos:

- Si la bala impacta un Asteroide, este se destruye, suma 1 punto y genera dos fragmentos desviados 45° respecto a la trayectoria del proyectil.
- Si la bala impacta un Fragmento, este se destruye directamente y suma 1 punto.

2.2 Reglas de Interacción y Físicas

- **Ausencia de Gravedad:** para no desviarse hacia el límite inferior de la pantalla, los asteroides, así como sus fragmentos resultantes, operan con *Use Gravity = false*
- **Gestión del Tiempo:** la interacción con el menú de pausa modifica globalmente la escala de tiempo (*Time.timeScale = 0f* o *1f*), interrumpiendo todas las traslaciones cinemáticas.

3. ASSETS Y GAMEOBJECTS

3.1 Inventario de GameObjects

Entidad / Prefab	Componentes	Constraints	Tag
Nave	<i>SpriteRenderer, CapsuleCollider, Rigidbody, Player.cs</i>	Freeze Pos Z, Freeze Rot X/Y/Z	Player
Asteroid	<i>SpriteRenderer, SphereCollider, Rigidbody, AsteroidFragmentation.cs</i>	Freeze Pos Z, Freeze Rot X/Y/Z	Enemy
Fragment	<i>SpriteRenderer, SphereCollider, Rigidbody, FragmentMovement.cs</i>	Freeze Pos Z, Freeze Rot X/Y/Z	Enemy
Bullet	<i>SpriteRenderer, BoxCollider, Rigidbody, Bullet.cs</i>	Freeze Pos Z	Untagged

3.2 Estructura de Managers

- **SpawnerManager:** Contiene el script generador de enemigos.
- **PoolManager:** Almacena los proyectiles inactivos para evitar instanciaciones masivas que degraden el rendimiento.
- **GameManager:** Contiene el *PauseManager.cs* y está vinculado mediante eventos *OnClick()* a la interfaz gráfica.

4. ARQUITECTURA DE SCRIPTS

4.1 Lógica de Enemigos y Generación

- **Asteroid.cs:** Gestiona la instanciación de amenazas. Extrae las dimensiones de la cámara principal para calcular posiciones de spawn fuera de la vista del

jugador. Traza un vector direccional matemáticamente restando la posición de aparición a la posición actual del Player

- **AsteroidFragmentation.cs:** Controla la reacción al impacto. Recibe el vector de dirección del asteroide padre y utiliza cuaterniones para calcular dos vectores de salida desviados 45 y -45 grados. Instancia el prefab del fragmento y le inyecta esta nueva trayectoria.
- **FragmentMovement.cs:** Otorga movimiento cinemático constante a los fragmentos soltados por un asteroide, recibiendo su trayectoria. Se autodestruye tras 8 segundos para optimizar memoria.

4.2 Lógica de Projectiles y Sistema

- **Bullet.cs:** Gestiona su propio ciclo de vida. Utiliza un contador de tiempo (*currentLifeTime*) frente a su vida máxima. En el evento *OnCollisionEnter*, verifica si choca con un *GameObject* con etiqueta *Enemy*. Si el enemigo tiene el componente *AsteroidFragmentation*, ejecuta el método *Shatter()*. De lo contrario, lo destruye instantáneamente. Tras el impacto, devuelve la bala al gestor de reciclaje.
- **PauseManager.cs:** Intercepta la tecla *Escape* para activar el Canvas *pauseMenuUI*.